

Orquestación de Contenedores: Introducción a Kubernetes y su Arquitectura

Kubernetes es la plataforma estándar de la industria para la **orquestación de contenedores**, esencial para desplegar, escalar y gestionar aplicaciones en entornos distribuidos. Aunque es una plataforma vasta y compleja, este capítulo se centra en su introducción, sus capacidades clave y su arquitectura fundamental.

I. Introducción y Características de Kubernetes

Kubernetes, desarrollado originalmente por Google y mantenido por la Cloud Native Computing Foundation (CNCF), es una plataforma de código abierto diseñada para automatizar la gestión de aplicaciones en contenedores.

1. Abstracción y Orquestación de Contenedores

Kubernetes proporciona una infraestructura centrada en contenedores y abstrae la infraestructura subyacente. Esto permite a los desarrolladores centrarse en la construcción y el despliegue de sus aplicaciones sin preocuparse por la gestión de la infraestructura.

- **Orquestación de Contenedores (*Container Orchestration*):** Automatiza el despliegue, escalado y gestión de aplicaciones en un clúster de máquinas. Programa contenedores en Nodos basándose en requisitos de recursos, restricciones y políticas definidas por el usuario.

2. Características Clave

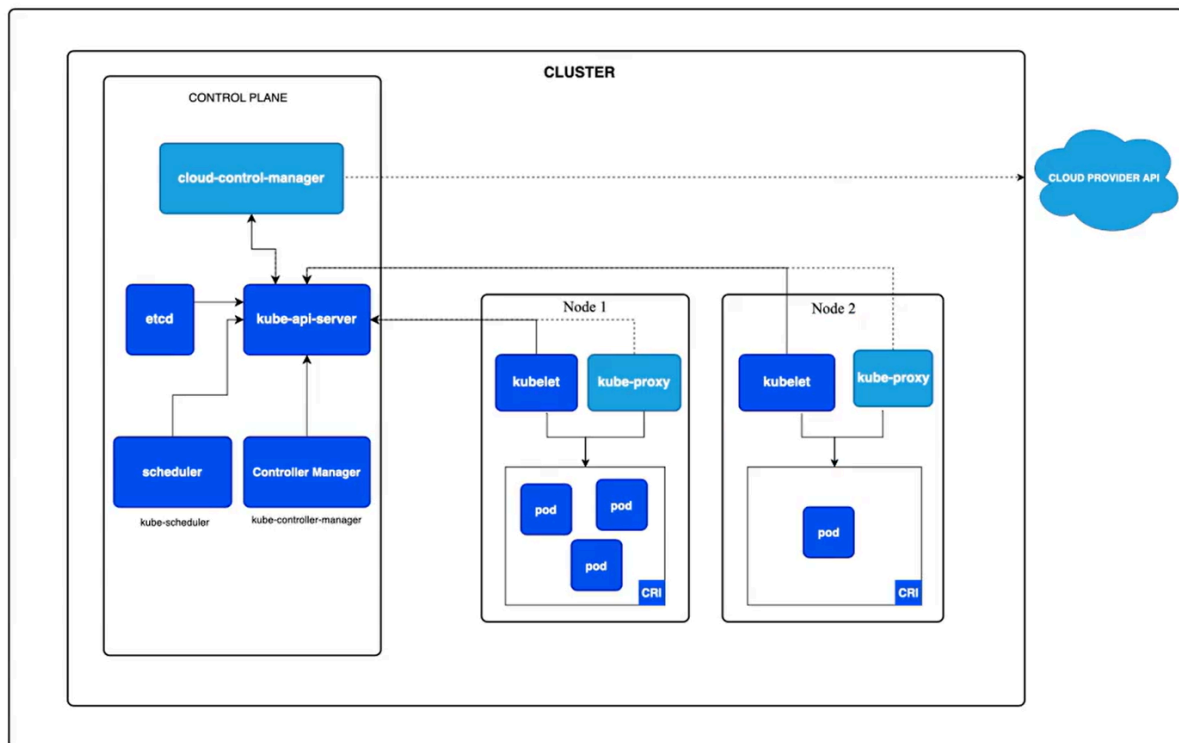
- **Descubrimiento de Servicios y Balanceo de Carga (*Service Discovery and Load Balancing*):** Proporciona mecanismos integrados para estas funcionalidades. Asigna una dirección IP única y un nombre DNS a cada

servicio en el clúster, facilitando la comunicación entre servicios sin necesidad de direcciones IP codificadas.

- **Escalado Automático (*Automatic Scaling*):** Puede escalar automáticamente aplicaciones en función de la utilización de CPU o memoria, permitiendo manejar cargas de trabajo variables de manera eficiente.
- **Autorreparación (*Self-Healing*):** Supervisa la salud de los contenedores y Nodos. Si un contenedor o Nodo falla, Kubernetes reinicia o reprograma automáticamente los componentes fallidos para garantizar una alta disponibilidad y confiabilidad.
- **Actualizaciones Graduales y Reversiones (*Rolling Updates and Rollbacks*):**
 - Admite **actualizaciones graduales** (*rolling updates*), implementando nuevas versiones sin tiempo de inactividad al reemplazar gradualmente las instancias antiguas por las nuevas.
 - Admite **reversiones** (*rollbacks*) a versiones anteriores si una actualización falla.
- **Orquestación de Almacenamiento (*Storage Orchestration*):** Proporciona una interfaz unificada para administrar volúmenes de almacenamiento. Ofrece varias soluciones de almacenamiento, incluido almacenamiento local, almacenamiento conectado en red y almacenamiento en la nube.
- **Gestión de Configuraciones (*Configurations Management*):** Permite definir configuraciones (como variables de entorno, secretos y archivos de configuración) utilizando objetos de Kubernetes como **ConfigMap** o **Secrets**. Estas configuraciones pueden inyectarse en contenedores en tiempo de ejecución.
- **Multitenencia y Aislamiento de Recursos (*Multi-tenancy and Resource Isolation*):** Permite que varios equipos o usuarios compartan el mismo clúster, garantizando el aislamiento de recursos y la seguridad entre cargas de trabajo.
- **Extensibilidad y Ecosistema:** Kubernetes es altamente extensible, permitiendo la personalización a través de su API, definiciones de recursos personalizados y Operadores.

II. Arquitectura de Kubernetes

Kubernetes está diseñado para gestionar y coordinar contenedores en un entorno distribuido y se compone de dos tipos de Nodos: el **Control Plane** (Nodo Maestro) y los **Nodos Trabajadores** (Worker Nodes).



1. Control Plane (Nodo Maestro)

El Control Plane es el componente central que gestiona el clúster y coordina las actividades.

- **kube-apiserver:** Es el componente central del Control Plane. Proporciona una interfaz *RESTful* a través de la cual se pueden gestionar y controlar los recursos de Kubernetes.
- **etcd:** Es el almacén de datos distribuido utilizado para almacenar la configuración y el estado del clúster. Puede existir en el Nodo Maestro o externalizarse.
- **kube-scheduler (scheduler):** Es el componente encargado de asignar **Pods** (las unidades de despliegue básicas) a los Nodos, basándose en los recursos disponibles y los requisitos de los Pods.

- **kube-controller-manager (*Controller Manager*)**: Es un conjunto de controladores que realizan tareas a nivel de clúster, como la gestión de Nodos, la replicación y la gestión del estado del clúster.
- **cloud-controller-manager**: Se utiliza para integrar Kubernetes con un proveedor de servicios en la nube (ej. AWS, Azure, Google Cloud). Proporciona funcionalidades específicas de la nube, como el aprovisionamiento de volúmenes y el balanceo de carga. Se comunica con la API del proveedor de la nube (*Cloud Provider API*).

2. Nodos Trabajadores (*Worker Nodes*)

Los Nodos Trabajadores son las máquinas donde se ejecutan realmente los contenedores de las aplicaciones.

- **kubelet**: Es el agente que se ejecuta en cada Nodo (tanto trabajadores como maestros). Se encarga de comunicarse con el *kube-apiserver* y gestiona los contenedores de Pods en el Nodo.
- **kube-proxy**: Es el responsable de gestionar el tráfico de red de los servicios de Kubernetes. Implementa la funcionalidad de balanceo de carga y reenvío de paquetes para los servicios.
- **Container Runtime (CRI)**: Es el motor de ejecución de contenedores (ej. *containerd* o Docker Engine). Es el responsable de ejecutar los contenedores en los Nodos.

III. Tipos de Clústeres

Existen diferentes formas de configurar y acceder a un clúster de Kubernetes:

1. **Clústeres en la Nube (*Cloud Clusters*)**: Servicios gestionados ofrecidos por los principales proveedores:
 - **GKE**: Google Kubernetes Engine (Google).
 - **EKS**: Elastic Kubernetes Service (Amazon Web Services).
 - **AKS**: Azure Kubernetes Service (Microsoft).
 - Otros proveedores como DigitalOcean también ofrecen sus propios servicios.
2. **Clústeres Locales**: Utilizados para desarrollo, pruebas y aprendizaje:

- **Docker Desktop:** Incluye la opción de levantar un clúster de Kubernetes localmente.
 - **Minikube:** Un clúster diseñado para el uso local por desarrolladores.
 - **Kind:** Un clúster local que se ejecuta encima de contenedores, a menudo utilizado para el propio desarrollo de Kubernetes.
3. **Clústeres Propios (*On-Premises*):** Herramientas para el despliegue de clústeres de Kubernetes en centros de datos propios o entornos privados.

IV. Conclusiones

Kubernetes es el orquestador dominante en el mercado de contenedores, con un ecosistema vasto y una comunidad enorme detrás. Se considera el "Sistema Operativo de la nube" y lanza nuevas versiones cada seis meses. Su arquitectura distribuida y sus capacidades de *self-healing* y escalado lo convierten en la plataforma *de facto* para el despliegue de aplicaciones modernas en contenedores.

Orquestación Local con Minikube: Configuración y Uso

Minikube es la herramienta *de facto* para configurar un entorno de **Kubernetes local** de manera ligera y sencilla. Está diseñado para simplificar el desarrollo, las pruebas y el aprendizaje de Kubernetes sin necesidad de acceder a un clúster de producción a gran escala.

I. Conceptos Fundamentales de Minikube

Minikube permite ejecutar un clúster de Kubernetes en una máquina local, encapsulando la complejidad del entorno distribuido.

- **Entorno Local de Kubernetes:** Proporciona una forma ligera y fácil de usar para configurar un clúster local, ideal para propósitos de desarrollo, pruebas y aprendizaje.
- **Clúster de Nodo Único (*Single-Node Cluster*):** Por defecto, Minikube crea clústeres de un solo nodo, donde **todos los componentes de Kubernetes** (Control Plane y Nodo Trabajador) se ejecutan dentro de ese mismo nodo. No obstante, es posible configurar clústeres de múltiples nodos.
- **Virtualización y Contenedores:** Minikube utiliza software de virtualización (como **VirtualBox** o **VMware**) o **contenedores Docker** para crear el clúster local. Si detecta Docker instalado, suele crear el clúster en contenedores.
- **Instalación Sencilla:** Escrito en un lenguaje compilado como **Go**, Minikube se puede instalar y ejecutar fácilmente en varios sistemas operativos (Linux, macOS, Windows) sin dependencias adicionales.
- **Paridad de Características (*Feature Parity*):** Su objetivo es mantener la paridad de funciones con los clústeres de Kubernetes en producción, permitiendo a los desarrolladores probar sus aplicaciones en un entorno que se asemeja fielmente al de producción.

- **Add-Ons y Extensiones:** Minikube soporta varios *add-ons* (plugins) que facilitan la habilitación de funciones adicionales, como el panel de control (*Dashboard*) de Kubernetes, soluciones de red y proveedores de almacenamiento.
-

II. Comandos en minikube:

Comandos Básicos (Basic Commands)

1. **start**

- **Explicación:** Inicia un clúster local de Kubernetes.
- **Sintaxis:** `minikube start`

Ejemplo: Para iniciar tu clúster con 4 GB de memoria:

```
minikube start --memory 4096
```

2. **status**

- **Explicación:** Obtiene el estado de un clúster local de Kubernetes (si está corriendo, si el host y el *kubelet* están disponibles, etc.).
- **Sintaxis:** `minikube status`

Ejemplo: Para verificar el estado actual del clúster:

```
minikube status
```

3. **stop**

- **Explicación:** Detiene un clúster local de Kubernetes en ejecución sin eliminar su configuración ni sus datos.
- **Sintaxis:** `minikube stop`

Ejemplo: Para detener el clúster:

```
minikube stop
```

4. **delete**

- **Explicación:** Elimina un clúster local de Kubernetes. Esto borra la máquina virtual, el *kubeconfig* y todos los datos asociados.
- **Sintaxis:** `minikube delete`

Ejemplo: Para eliminar completamente el clúster:

```
minikube delete
```

5. **dashboard**

- **Explicación:** Accede al *dashboard* de Kubernetes que se ejecuta dentro del clúster de minikube, abriéndolo en tu navegador.
- **Sintaxis:** `minikube dashboard`

Ejemplo: Para abrir el *dashboard*:

```
minikube dashboard
```

6. **pause**

- **Explicación:** Pausa Kubernetes, deteniendo todos los contenedores y procesos del clúster sin detener la máquina virtual subyacente.
- **Sintaxis:** `minikube pause`

Ejemplo: Para pausar temporalmente todos los *workloads*:

```
minikube pause
```

7. **unpause**

- **Explicación:** Despausa Kubernetes, reanudando las operaciones del clúster después de un comando `pause`.
- **Sintaxis:** `minikube unpause`

Ejemplo: Para reanudar el clúster:

```
minikube unpause
```

Comandos de Imágenes (Images Commands)

1. **docker-env**

- **Explicación:** Proporciona instrucciones para apuntar el CLI de Docker de tu terminal al *Docker Engine* que se ejecuta dentro de minikube. Esto es útil para construir imágenes de Docker directamente dentro de minikube.
- **Sintaxis:** `minikube docker-env`

Ejemplo: Para configurar tu *shell* actual (Bash):

```
eval $(minikube docker-env)
```

2. **podman-env**

- **Explicación:** Configura el entorno para usar el servicio *Podman* de minikube.
- **Sintaxis:** `minikube podman-env`
- **Ejemplo:** Para usar Podman dentro de minikube (el comando de configuración varía según la *shell*).

3. **cache**

- **Explicación:** Gestiona la caché de imágenes.
- **Sintaxis:** `minikube cache <comando>`

Ejemplo: Para listar las imágenes en la caché:

```
minikube cache list
```

4. **image**

- **Explicación:** Gestiona imágenes. Te permite cargar o eliminar imágenes en el entorno de minikube.
- **Sintaxis:** `minikube image <comando>`

Ejemplo: Para cargar una imagen de Docker local al clúster de minikube:

```
minikube image load mi-imagen:latest
```

Comandos de Configuración y Gestión (Configuration and Management Commands)

1. **addons**

- **Explicación:** Habilita o deshabilita un complemento (*addon*) de minikube (como *metrics-server* o *ingress*).
- **Sintaxis:** `minikube addons <comando>`

Ejemplo: Para habilitar el complemento de *ingress*:

```
minikube addons enable ingress
```

2. **config**

- **Explicación:** Modifica valores de configuración persistentes para minikube.
- **Sintaxis:** `minikube config <comando>`

Ejemplo: Para ver el *driver* actual:

```
minikube config get driver
```

3. **profile**

- **Explicación:** Permite obtener o listar los perfiles de clúster (o clústeres) actuales. Útil si ejecutas múltiples clústeres de minikube.
- **Sintaxis:** `minikube profile <comando>`

Ejemplo: Para listar todos los perfiles disponibles:

```
minikube profile list
```

4. **update-context**

- **Explicación:** Actualiza el archivo *kubeconfig* en caso de un cambio de IP o puerto del clúster.
- **Sintaxis:** `minikube update-context`

Ejemplo: Para asegurar que tu *kubeconfig* esté actualizado:

```
minikube update-context
```

Comandos de Red y Conectividad (Networking and Connectivity Commands)

1. `service`

- **Explicación:** Devuelve una URL para conectarse a un servicio específico de Kubernetes.
- **Sintaxis:** `minikube service <nombre_del_servicio>`

Ejemplo: Para obtener la URL del servicio `mi-app`:

```
minikube service mi-app
```

2. `tunnel`

- **Explicación:** Conecta servicios de tipo `LoadBalancer` (Balanceador de Carga) al *host* local, usando un túnel.
- **Sintaxis:** `minikube tunnel`

Ejemplo: Para iniciar el túnel de balanceador de carga (debe ejecutarse en una terminal separada):

```
minikube tunnel
```

Comandos Avanzados (Advanced Commands)

1. `mount`

- **Explicación:** Monta el directorio especificado del *host* dentro del entorno de minikube.
- **Sintaxis:** `minikube mount <ruta_host>:<ruta_minikube>`

Ejemplo: Para montar el directorio de trabajo local:

```
minikube mount $(pwd):/host_data
```

2. `ssh`

- **Explicación:** Inicia una sesión SSH para ingresar al entorno de la máquina virtual de minikube (útil para depuración).

- **Sintaxis:** `minikube ssh`

Ejemplo: Para acceder a la máquina virtual:

`minikube ssh`

3. `kubectl`

- **Explicación:** Ejecuta un binario de `kubectl` que coincide con la versión del clúster, asegurando la compatibilidad.
- **Sintaxis:** `minikube kubectl -- <comando_kubectl>`

Ejemplo: Para ver los *namespaces*:

`minikube kubectl -- get namespaces`

4. `node`

- **Explicación:** Permite añadir, eliminar o listar nodos adicionales al clúster (para simular un clúster multi-nodo).
- **Sintaxis:** `minikube node <comando>`

Ejemplo: Para añadir un segundo nodo:

`minikube node add`

5. `cp`

- **Explicación:** Copia el archivo especificado dentro del entorno de minikube.
- **Sintaxis:** `minikube cp <archivo>:<ruta_destino>`

Ejemplo: Para copiar un archivo de certificado al clúster:

`minikube cp cert.pem /home/docker/`



Comandos de Solución de Problemas (Troubleshooting Commands)

1. `ssh-key`

- **Explicación:** Recupera la ruta de la clave de identidad SSH del nodo especificado.
- **Sintaxis:** `minikube ssh-key`
- **Ejemplo:** Para ver la ruta de la clave SSH.

2. `ssh-host`

- **Explicación:** Recupera el *host* SSH del nodo especificado.
- **Sintaxis:** `minikube ssh-host`
- **Ejemplo:** Para obtener la dirección del host SSH.

3. `ip`

- **Explicación:** Recupera la dirección IP del nodo especificado.
- **Sintaxis:** `minikube ip`

Ejemplo: Para conocer la IP del clúster:

```
minikube ip
```

4. `logs`

- **Explicación:** Devuelve los logs para depurar un clúster local de Kubernetes.
- **Sintaxis:** `minikube logs`

Ejemplo: Para ver los logs del sistema del clúster:

```
minikube logs
```

5. `update-check`

- **Explicación:** Imprime el número de versión actual y el número de la última versión disponible.
- **Sintaxis:** `minikube update-check`
- **Ejemplo:** Para verificar si hay una nueva versión.

6. `version`

- **Explicación:** Imprime la versión de minikube instalada.
- **Sintaxis:** `minikube version`

Ejemplo: Para mostrar la versión actual:

`minikube version`

7. **options**

- **Explicación:** Muestra una lista de opciones globales de línea de comandos que se aplican a todos los comandos.
 - **Sintaxis:** `minikube options`
 - **Ejemplo:** Para ver todas las opciones globales.
-

Otros Comandos (Other Commands)

1. **completion**

- **Explicación:** Genera la finalización de comandos para una *shell* (como Bash o Zsh).
- **Sintaxis:** `minikube completion <shell>`

Ejemplo: Para configurar la finalización en Bash:

`minikube completion bash`

2. **license**

- **Explicación:** Muestra las licencias de las dependencias en un directorio.
 - **Sintaxis:** `minikube license`
 - **Ejemplo:** Para ver la información de la licencia.
-

III. Instalación y Gestión Práctica de Minikube

La instalación de Minikube es directa, generalmente a través de la descarga e instalación de su binario en el sistema *host*.

Paso 1: Instalación del Binario de Minikube

1. **Descarga del Binario:** Se utiliza `curl` para descargar el binario de Minikube adecuado para el sistema operativo y la arquitectura.

```
curl -LO
https://storage.googleapis.com/minikube/releases/latest/m
inikube-linux-amd64
```

2. **Instalación:** Se instala el binario en una ruta accesible por el sistema (ej. `/usr/local/bin`).

```
sudo install minikube-linux-amd64 /usr/local/bin/minikube
```

Paso 2: Inicio y Configuración del Clúster (`minikube start`)

El comando `minikube start` inicializa el clúster. Minikube analiza el *host* y decide el *driver* de virtualización (contenedores, VirtualBox, etc.) a utilizar.

1. **Inicio del Clúster:**

```
minikube start
```

- *Proceso:* Minikube descarga primero la imagen base de Kubernetes que se instalará en el nodo. Luego, crea un contenedor Docker o una máquina virtual, asignándole recursos por defecto (ej. **2 CPUs y 2 GB de memoria**). Estos recursos son configurables por el usuario, siempre y cuando estén disponibles en el sistema.
2. **Verificación del Estado:** El comando `minikube status` proporciona información sobre el estado de los componentes del clúster.

```
minikube status
```

- *Resultado:* El estado confirmará que el *Control Plane*, el *kubelet*, el *host* y el *API server* están funcionando, así como la configuración del `kubeconfig`.

Paso 3: Gestión y Diagnóstico del Clúster

Minikube proporciona comandos sencillos para el manejo del clúster local.

1. **Acceso SSH al Nodo Único:** El comando `minikube ssh` permite acceder directamente a la *shell* del nodo (que es un contenedor Docker o una VM) donde se está ejecutando el clúster de Kubernetes.

```
minikube ssh
```

2. **Gestión de Add-Ons (Plugins):** Minikube facilita la habilitación de funcionalidades complejas a través de *add-ons*.

```
minikube addons list
```

- *Propósito:* Muestra una lista de todos los plugins disponibles (como **dashboard**, *ingress*, soporte para **GPU**). Aunque estas funciones se pueden configurar manualmente, los *plugins* simplifican enormemente el proceso.
3. **Ciclo de Vida del Clúster:** Minikube ofrece comandos para el control del clúster:
 - `minikube stop`: Para detener el clúster.
 - `minikube delete`: Para eliminar completamente el clúster.

III. Conclusiones

Minikube es una solución ideal para probar y experimentar con Kubernetes de forma local. Al ser lo más parecido a un clúster de producción, es perfecto para el desarrollo y las pruebas de aplicaciones. Cuenta con un gran soporte y una amplia

comunidad, lo que lo convierte en la solución idónea para interactuar con Kubernetes a nivel individual.

Gestión y Acceso a Kubernetes: Cliente CLI

(**kubectl**) y Dashboard

Una vez que un clúster de **Kubernetes** está operativo (como con **Minikube**), es necesario utilizar una herramienta cliente para interactuar con él, gestionar recursos y desplegar aplicaciones. Existen dos vías principales: la línea de comandos (CLI), que es la herramienta oficial, y las interfaces gráficas (Dashboard).

I. Introducción a los Clientes de Kubernetes

El cliente de Kubernetes actúa como el punto de control entre el usuario y el **kube-apiserver** del clúster.

1. Cliente de Línea de Comandos (CLI)

- **Nombre Oficial:** **kubectl**
- **Rol:** Es el cliente oficial y más utilizado. Ofrece la máxima flexibilidad, potencia y es indispensable para la automatización, el *scripting* y el trabajo en entornos remotos (como en *pipelines* o servidores).

2. Cliente Gráfico (GUI)

- **Ejemplo:** El *Dashboard* original de Kubernetes.
- **Rol:** Proporciona una interfaz visual (*mouse-and-click*) para el monitoreo y la gestión básica del clúster.

II. Instalación y Uso del Cliente Oficial (**kubectl**)

La instalación de **kubectl** implica descargar el binario y hacerlo ejecutable en el *host*.

Procedimiento de Instalación de **kubectl** en Linux

1. **Descarga del Binario:** Se utiliza **curl** para descargar el binario de **kubectl** desde la fuente oficial de Kubernetes, acorde a la arquitectura del sistema.

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kub
ectl"
```

2. **Verificación del Binario (*Checksum*):** Aunque se omitió en la demostración, es una práctica de seguridad crucial **comprobar el *checksum*** del binario descargado (utilizando archivos de suma de comprobación proporcionados en la página oficial) para asegurar la integridad del archivo y que no ha sido alterado.
3. **Instalación:** Se utiliza `sudo install` para mover el binario a una ruta accesible por el sistema (ej. `/usr/local/bin`), donde puede ser ejecutado por cualquier usuario.

```
sudo install kubectl /usr/local/bin/kubectl
```

Sintaxis Inversa de `kubectl`

A diferencia de Docker, que sigue el formato `docker [RECURSO] [VERBO]`, `kubectl` utiliza un formato inverso:

- **Formato Docker:** `docker container ls`
- **Formato Kubernetes (Inverso):** `kubectl [VERBO] [RECURSO]`

Elemento	Sintaxis	Ejemplo	Propósito
Comando	<code>kubectl</code>	<code>kubectl</code>	El cliente de control.
Verbo	<code>get</code>	<code>get</code>	La acción a realizar (obtener, crear, eliminar).
Recurso	<code>pods</code>	<code>pods</code>	El objeto de Kubernetes sobre el que actúa (Pod, Service, Deployment).

Ejemplo Práctico: Para listar los Pods en el clúster:

```
kubectl get pods
```

III. Acceso y Uso del Dashboard (GUI)

Minikube facilita el acceso a la interfaz gráfica de Kubernetes a través de un *add-on*.

1. Habilitación del Add-on

El *add-on* de `dashboard` no está habilitado por defecto y debe instalarse en el clúster.

1. **Listar Add-ons:** Se verifica el estado de los *plugins* de Minikube.

```
minikube addons list
```

2. **Habilitar Dashboard:** Se utiliza el comando `minikube addons enable`.

```
minikube addons enable dashboard
```

- a. *Resultado:* Esto instala el *Dashboard* dentro del clúster de Minikube.

2. Lanzamiento del Dashboard

El comando `minikube dashboard` abre automáticamente la interfaz gráfica en el navegador del *host*.

- **Sintaxis:**

```
minikube dashboard
```

- **Funcionalidad:** La GUI permite visualizar y gestionar recursos de forma interactiva:
 - **Workloads:** Las aplicaciones en ejecución (Deployments, Pods, etc.).

- **Services:** Los servicios de exposición y red.
- **Configuration & Storage:** Objetos como ConfigMaps y volúmenes persistentes.
- **Namespaces:** La vista de los diferentes espacios de nombres (ej. `kube-system`).

V. Comandos de `kubectl` con Sintaxis y Ejemplos

Comandos Básicos (Principiantes e Intermedios)

1. `create`

- **Explicación:** Crea un recurso de Kubernetes a partir de un archivo o de la entrada estándar (`stdin`).
- **Sintaxis:** `kubectl create -f <archivo> | -f -`
- **Ejemplo de Uso:** Para crear un *pod* desde un archivo: `kubectl create -f pod.yaml`

2. `expose`

- **Explicación:** Toma un *replication controller*, un servicio, un *deployment* o un *pod* y lo expone como un nuevo servicio de Kubernetes.
- **Sintaxis:** `kubectl expose <tipo> <nombre> --port=<puerto_servicio>`
- **Ejemplo de Uso:** Exponer el *deployment* `nginx-app` en el puerto 80: `kubectl expose deployment nginx-app --port=80`

3. `run`

- **Explicación:** Ejecuta una imagen particular en el clúster, creando un *pod* o un *deployment*.
- **Sintaxis:** `kubectl run <nombre> --image=<imagen> [options]`

- **Ejemplo de Uso:** Ejecutar una imagen de BusyBox: `kubectl run debug-box --image=busybox --restart=Never -- sleep 3600`

4. **set**

- **Explicación:** Establece características específicas en los objetos, como actualizar la imagen o las variables de entorno.
- **Sintaxis:** `kubectl set <tipo_de_objeto> <nombre> --<parámetro>=<valor>`
- **Ejemplo de Uso:** Cambiar la imagen del *deployment* `mi-app`:
`kubectl set image deployment/mi-app mi-app=nueva-imagen:v2`

5. **explain**

- **Explicación:** Obtiene la documentación (estructura y campos) para un recurso de Kubernetes.
- **Sintaxis:** `kubectl explain <recurso>`
- **Ejemplo de Uso:** Ver la documentación para el recurso `Service`:
`kubectl explain service`

6. **get**

- **Explicación:** Muestra uno o muchos recursos (ej. *Pods*, *deployments*).
- **Sintaxis:** `kubectl get <recurso> [<nombre>]`
- **Ejemplo de Uso:** Listar todos los *Pods* en el *namespace* actual en formato ancho: `kubectl get pods -o wide`

7. **edit**

- **Explicación:** Edita un recurso en el servidor abriendo un editor de texto.
- **Sintaxis:** `kubectl edit <recurso> <nombre>`
- **Ejemplo de Uso:** Editar el *deployment* `mi-app`: `kubectl edit deployment mi-app`

8. **delete**

- **Explicación:** Elimina recursos por nombres, archivos o selectores de etiquetas.
 - **Sintaxis:** `kubectl delete <recurso> <nombre> | -f <archivo>`
 - **Ejemplo de Uso:** Eliminar todos los *Pods* con la etiqueta `app=temporal`: `kubectl delete pods -l app=temporal`
-

Comandos de Despliegue (Deploy Commands)

9. **rollout**

- **Explicación:** Gestiona el despliegue (lanzamiento o retroceso) de un recurso.
- **Sintaxis:** `kubectl rollout <subcomando> <recurso>/<nombre>`
- **Ejemplo de Uso:** Deshacer el último *rollout* de un *deployment*:
`kubectl rollout undo deployment/mi-app`

10. **scale**

- **Explicación:** Establece un nuevo tamaño (número de réplicas) para un *deployment*, *replica set* o *replication controller*.
- **Sintaxis:** `kubectl scale <recurso>/<nombre> --replicas=<N>`
- **Ejemplo de Uso:** Escalar el *deployment* `mi-app` a 5 réplicas:
`kubectl scale deployment/mi-app --replicas=5`

11. **autoscale**

- **Explicación:** Configura el autoescalado horizontal para un recurso (*deployment*, *replica set*, etc.).
- **Sintaxis:** `kubectl autoscale <recurso>/<nombre> --min=<N> --max=<M> --cpu-percent=<P>`

- **Ejemplo de Uso:** Configurar autoescalado para `mi-app`: `kubectl autoscale deployment mi-app --min=2 --max=10 --cpu-percent=70`
-

Comandos de Gestión de Clúster (Cluster Management Commands)

12. `certificate`

- **Explicación:** Modifica recursos de certificado (*CertificateSigningRequest* - CSR).
- **Sintaxis:** `kubectl certificate <subcomando> <nombre>`
- **Ejemplo de Uso:** Aprobar una solicitud de firma de certificado:
`kubectl certificate approve my-csr`

13. `cluster-info`

- **Explicación:** Muestra información del clúster (ubicación del *control plane* y servicios).
- **Sintaxis:** `kubectl cluster-info`
- **Ejemplo de Uso:** `kubectl cluster-info`

14. `top`

- **Explicación:** Muestra el uso de recursos (CPU/memoria) por parte de los nodos o *pods*.
- **Sintaxis:** `kubectl top <recurso>`
- **Ejemplo de Uso:** Ver el uso de CPU y memoria de los nodos:
`kubectl top node`

15. `cordon`

- **Explicación:** Marca un nodo como **no planificable** (*unschedulable*), evitando que se le asignen nuevos *pods*.
- **Sintaxis:** `kubectl cordon <nombre_del_nodo>`
- **Ejemplo de Uso:** `kubectl cordon node-1`

16. `uncordon`

- **Explicación:** Marca un nodo como **planificable** (*schedulable*), revirtiendo la acción de *cordon*.
- **Sintaxis:** `kubectl uncordon <nombre_del_nodo>`
- **Ejemplo de Uso:** `kubectl uncordon node-1`

17. **drain**

- **Explicación:** Vacía un nodo, **expulsando** de forma segura todos los *pods* en preparación para mantenimiento.
- **Sintaxis:** `kubectl drain <nombre_del_nodo>`
- **Ejemplo de Uso:** `kubectl drain node-1
--ignore-daemonsets`

18. **taint**

- **Explicación:** Actualiza las *taints* (manchas) en uno o más nodos, utilizado para la afinidad o rechazo de programación de *pods*.
- **Sintaxis:** `kubectl taint node <nombre_del_nodo>
<clave>=<valor>:<efecto>`
- **Ejemplo de Uso:** `kubectl taint node node-1
key=value:NoExecute`

Comandos de Depuración y Solución de Problemas

19. **describe**

- **Explicación:** Muestra detalles extensos, estado y eventos recientes de un recurso específico.
- **Sintaxis:** `kubectl describe <recurso> <nombre>`
- **Ejemplo de Uso:** `kubectl describe pod mi-pod-xyz`

20. **logs**

- **Explicación:** Imprime los *logs* de un contenedor en un *pod*.
- **Sintaxis:** `kubectl logs [-f] <nombre_del_pod> [-c
<nombre_del_contenedor>]`

- **Ejemplo de Uso:** Ver los *logs* en tiempo real (*-f*) de un contenedor:
`kubectl logs -f mi-pod-xyz`

21. **attach**

- **Explicación:** Se adjunta a un contenedor en ejecución para interactuar con él (útil para ver la salida del proceso principal).
- **Sintaxis:** `kubectl attach <nombre_del_pod> -c <nombre_del_contenedor>`
- **Ejemplo de Uso:** `kubectl attach mi-pod-xyz -c app-container`

22. **exec**

- **Explicación:** Ejecuta un comando dentro de un contenedor en un *pod*.
- **Sintaxis:** `kubectl exec -it <nombre_del_pod> -- <comando>`
- **Ejemplo de Uso:** Abrir una *shell* en el contenedor: `kubectl exec -it mi-pod-xyz -- bash`

23. **port-forward**

- **Explicación:** Reenvía uno o más puertos locales a un *pod* o servicio.
- **Sintaxis:** `kubectl port-forward <recurso>/<nombre> <puerto_local>:<puerto_remoto>`
- **Ejemplo de Uso:** Acceder al puerto 80 del *pod* en el puerto 8080 local:
`kubectl port-forward pod/mi-pod-xyz 8080:80`

24. **proxy**

- **Explicación:** Ejecuta un *proxy* entre el *host* local y el servidor API de Kubernetes.
- **Sintaxis:** `kubectl proxy --port=<puerto_local>`
- **Ejemplo de Uso:** Iniciar el *proxy* para acceso a la API: `kubectl proxy --port=8001`

25. **cp**

- **Explicación:** Copia archivos y directorios hacia y desde contenedores.
- **Sintaxis:** `kubectl cp <origen> <destino>`

- **Ejemplo de Uso:** Copiar un archivo local al *pod*: `kubectl cp config.json mi-pod-xyz:/app/`

26. **auth**

- **Explicación:** Inspecciona la autorización, verificando los permisos de un usuario o cuenta de servicio.
- **Sintaxis:** `kubectl auth can-i <verbo> <recurso>`
- **Ejemplo de Uso:** Verificar si puedes listar *pods*: `kubectl auth can-i get pods`

27. **debug**

- **Explicación:** Crea sesiones de depuración para la solución de problemas de *workloads* y nodos.
- **Sintaxis:** `kubectl debug <recurso>/<nombre>`
- **Ejemplo de Uso:** Adjuntar una sesión de depuración a un *pod*:
`kubectl debug pod/mi-pod-fallido`

28. **events**

- **Explicación:** Lista los eventos (historial de actividades) en el clúster.
- **Sintaxis:** `kubectl events`
- **Ejemplo de Uso:** `kubectl events`

Comandos Avanzados y de Configuración (**Advanced Commands & Settings Commands**)

29. **diff**

- **Explicación:** Compara la versión en vivo de un recurso con la versión que sería aplicada por un archivo YAML.
- **Sintaxis:** `kubectl diff -f <archivo>`
- **Ejemplo de Uso:** `kubectl diff -f deployment.yaml`

30. **apply**

- **Explicación:** Aplica una configuración a un recurso a través de un archivo, creando o actualizando el recurso.

- **Sintaxis:** `kubectl apply -f <archivo>`
- **Ejemplo de Uso:** `kubectl apply -f deployment.yaml`

31. **patch**

- **Explicación:** Actualiza campos específicos de un recurso (modificación parcial).
- **Sintaxis:** `kubectl patch <recurso> <nombre> -p '<JSON o YAML patch>'`
- **Ejemplo de Uso:** `kubectl patch deployment mi-app -p '{"spec": {"replicas": 3}}'`

32. **replace**

- **Explicación:** Reemplaza completamente un recurso existente con un nuevo archivo de configuración.
- **Sintaxis:** `kubectl replace -f <archivo>`
- **Ejemplo de Uso:** `kubectl replace -f service-prod.yaml`

33. **wait**

- **Explicación:** Experimental: Espera una condición específica en uno o muchos recursos.
- **Sintaxis:** `kubectl wait --for=<condición> <recurso>/<nombre>`
- **Ejemplo de Uso:** Esperar hasta que un *deployment* esté disponible:
`kubectl wait --for=condition=Available deployment/mi-app --timeout=60s`

34. **kustomize**

- **Explicación:** Construye un objetivo de Kustomization a partir de un directorio o URL.
- **Sintaxis:** `kubectl kustomize <directorio>`
- **Ejemplo de Uso:** Generar el manifiesto final para producción:
`kubectl kustomize overlays/prod`

35. **label**

- **Explicación:** Actualiza las etiquetas (**labels**) en un recurso.

- **Sintaxis:** `kubectl label <recurso> <nombre>
<clave>=<valor>`
- **Ejemplo de Uso:** `kubectl label pod mi-pod-xyz
tier=frontend`

36. **annotate**

- **Explicación:** Actualiza las anotaciones (`annotations`) en un recurso.
- **Sintaxis:** `kubectl annotate <recurso> <nombre>
<clave>=<valor>`
- **Ejemplo de Uso:** `kubectl annotate deployment mi-app
"description"="App de prueba"`

37. **completion**

- **Explicación:** Genera el código de autocompletado de la *shell* para la *shell* especificada (*bash*, *zsh*, *fish* o *powershell*).
 - **Sintaxis:** `kubectl completion <shell>`
 - **Ejemplo de Uso:** Para configurar la finalización en Bash: `source
<(kubectl completion bash)`
-

Otros Comandos de Información (**Other Commands**)

38. **api-resources**

- **Explicación:** Imprime los recursos de API compatibles en el servidor.
- **Sintaxis:** `kubectl api-resources`
- **Ejemplo de Uso:** `kubectl api-resources`

39. **api-versions**

- **Explicación:** Imprime las versiones de API compatibles en el servidor, en el formato "grupo/versión".
- **Sintaxis:** `kubectl api-versions`
- **Ejemplo de Uso:** `kubectl api-versions`

40. **config**

- **Explicación:** Modifica archivos *kubeconfig* (para gestionar contextos y credenciales).
- **Sintaxis:** `kubectl config <subcomando>`
- **Ejemplo de Uso:** Cambiar al contexto `prod-cluster`: `kubectl config use-context prod-cluster`

41. **plugin**

- **Explicación:** Proporciona utilidades para interactuar con *plugins* de `kubectl`.
- **Sintaxis:** `kubectl plugin <subcomando>`
- **Ejemplo de Uso:** Listar los *plugins* instalados: `kubectl plugin list`

42. **version**

- **Explicación:** Imprime la información de la versión del cliente (`kubectl`) y del servidor (API del clúster).
- **Sintaxis:** `kubectl version`
- **Ejemplo de Uso:** `kubectl version`

V. Conclusiones y Elección del Cliente

- **Diferenciación:** Las interfaces gráficas (*Dashboard*) son cómodas de usar (*mouse-and-click*) y útiles para la inspección rápida.
- **Flexibilidad y Poder:** El cliente `kubectl` ofrece la **flexibilidad, el poder y la capacidad de automatización** que son fundamentales en un entorno de máster y en el trabajo profesional.
- **Recomendación:** Al igual que con Docker, la **línea de comandos (`kubectl`)** es la herramienta preferida para desarrollar *scripts*, automatizar procesos en *pipelines* de CI/CD o trabajar en remoto. Por lo tanto, las próximas sesiones se centrarán exclusivamente en la CLI.

Objetos Fundamentales en Kubernetes:

Namespaces y Pods

En Kubernetes, toda entidad gestionable es un **Objeto** o **Recurso**. El clúster organiza y aísla estos recursos utilizando **Namespaces**, y la unidad más pequeña de despliegue y computación es el **Pod**. Comprender estos fundamentos es esencial para gestionar y asegurar cualquier aplicación en Kubernetes.

I. Introducción a los Objetos de Kubernetes

Todo lo que se crea y gestiona en Kubernetes es un **Objeto** o **Recurso**, cuya definición es declarativa.

Campos Comunes de los Objetos

Todos los objetos en Kubernetes tienen una definición estándar, generalmente en formato **YAML** (o JSON), que incluye tres campos mandatorios y comunes:

1. **apiVersion**: Indica la versión de la API que se utilizará para definir ese recurso en particular (ej. **v1** para recursos fundamentales como Pods).
2. **kind**: Indica el tipo de recurso que se está definiendo (ej. **Pod**, **Service**, **Deployment**). La palabra clave *Kind* debe coincidir exactamente con el formato de la API (generalmente *CamelCase*, ej. **Pod** con P mayúscula).
3. **metadata**: Contiene metadatos sobre el objeto, siendo el campo **name** mandatorio. También se pueden añadir etiquetas (*labels*) para la organización y selección de recursos.

La sintaxis del cliente (**kubectl**) sigue el formato **Verbo** y **Recurso** (al revés que Docker): **kubectl [VERBO] [RECURSO]**.

- **Comando de Consulta de Recursos:** El comando `kubectl api-resources` muestra la lista completa de todos los recursos que existen en el clúster. La lista indica el `NAME` (plural), `SHORTNAMES` (abreviación), `APIVERSION` y si el recurso está `NAMESPACED` (si pertenece a un espacio de nombres).
-

II. Agrupación Lógica y Aislamiento: Namespaces

Un **Namespace** (espacio de nombres) es un mecanismo de **agrupación lógica** utilizado para organizar y segmentar objetos dentro de un clúster de Kubernetes.

Propósito y Características

- **Organización:** Permite que múltiples usuarios, equipos o aplicaciones compartan el mismo clúster sin interferir entre sí. Es similar a las carpetas en un sistema de ficheros, con la diferencia de que un *Namespace* no puede contener otro *Namespace*, solo recursos.
- **Default Namespace:** Los clústeres vienen con un espacio de nombres predeterminado llamado `default`. Si no se especifica uno, los comandos trabajan con este *Namespace*.
- **Control de Acceso (RBAC):** Kubernetes admite **Control de Acceso Basado en Roles (RBAC)** a nivel de *Namespace*. Esto permite a los administradores definir políticas detalladas sobre quién puede crear, ver, modificar o eliminar objetos dentro de un *Namespace* especificado.
- **Cuotas de Recursos:** Es posible aplicar **cuotas de recursos** a un *Namespace*, limitando la cantidad de CPU, memoria o almacenamiento que puede consumir. Esto aplica el mismo principio de limitación de recursos que vimos con Docker (`Cgroups`) para evitar que un *Namespace* consuma todos los recursos del clúster.

Procedimientos Prácticos con Namespaces

1. **Listar Namespaces:** Se utiliza el verbo `get` y se puede usar el nombre completo (`namespaces`), el singular (`namespace`) o la abreviación (`ns`).

```
kubectl get namespaces # Muestra los namespaces del clúster
```

- a. *Resultado:* Muestra el `default`, `kube-system` (donde Kubernetes instala sus recursos propios), y otros.

2. **Especificar un Namespace:** Para trabajar en un *Namespace* distinto al `default`, se utiliza el *flag* `-n`.

```
kubectl get pods -n kube-system
```

III. Unidad Básica de Despliegue: Pods

El **Pod** es la unidad más pequeña y básica de despliegue y computación en Kubernetes. Cualquier tarea que se ejecute en Kubernetes debe ejecutarse dentro de un Pod.

Estructura y Red de los Pods

- **Agrupación de Contenedores:** Un Pod representa una o más instancias de contenedores que están fuertemente acopladas y se ejecutan juntas.
- **Recursos Compartidos:** Todos los contenedores dentro de un mismo Pod **comparten el mismo entorno de red y almacenamiento**, incluyendo opciones de configuración como volúmenes y variables de entorno.
- **Dirección IP:** La dirección IP se asigna al **Pod** y no al contenedor individual. Esto significa que todos los contenedores dentro del Pod comparten la misma IP y el mismo espacio de red.
- **Conflicto de Puertos:** Debido a que los contenedores en el mismo Pod comparten la misma IP, **no pueden utilizar el mismo puerto**.

- **Volúmenes:** Los volúmenes que se almacenan dentro del propio Pod se conocen como **volúmenes efímeros**. Al igual que la capa de contenedor en Docker, los datos almacenados en estos volúmenes se destruyen cuando el Pod se destruye.



El esquema ilustra diferentes configuraciones de Pods:

- **Pod 1:** Contiene un único contenedor y se le asigna una IP (**10.10.10.1**).
- **Pod 2:** Contiene un contenedor y un volumen efímero.
- **Pod 3:** Contiene dos contenedores y un volumen efímero.
- **Pod 4:** Contiene tres contenedores y dos volúmenes, y se le asigna una IP (**10.10.10.4**).

Procedimientos Prácticos con Pods

1. **Definición Declarativa de Pods:** Se utiliza un archivo YAML para definir el Pod, especificando los campos comunes y la sección **spec** para los contenedores.

Ejemplo de Definición de Pod con dos contenedores:

```

apiVersion: v1

kind: Pod
metadata:
  name: first-pod
  labels:
    label1: value
spec:

```

```
containers:
  - name: nginx
    image: nginx
    ports:
      - containerPort: 80
        name: http
        protocol: TCP
    env:
      - name: MYVAR
        value: myvalue
  - name: container2
    image: alpine
```

- **Campos Clave en `spec`:** Se define un *array* de contenedores, cada uno con su `name`, `image` (la imagen Docker a usar, que Kubernetes descarga del registro si no existe en la caché), y la definición opcional de `ports` y `env` (variables de entorno).

2. **Despliegue Declarativo (`kubectl apply`):** La forma recomendada de desplegar recursos es la **forma declarativa** utilizando `kubectl apply`.

Kubernetes gestiona la creación o modificación del recurso.

- Si el objeto no existe, lo crea.
- Si el objeto existe, calcula las diferencias y aplica solo los cambios definidos en el archivo.

```
kubectl apply -f my-first-pod.yaml
```

```
# El flag -f indica la ruta al fichero.
```

3. **Inspección del Pod y la Red:** Se utiliza el *flag* `-o wide` para obtener información extendida, incluyendo la IP interna del Pod.

```
kubectl get po -o wide
```

4. Acceso al Pod (Debugging): La IP interna del Pod es inaccesible directamente desde el *host* fuera del clúster.

- **Opción 1: Acceso dentro del clúster (debugging):** Utilizar `minikube ssh` para entrar en el nodo del clúster y acceder a la IP del Pod con `curl`.
- **Opción 2: Creación de Túnel (desarrollo):** El comando `kubectl port-forward` crea un túnel temporal, mapeando un puerto del *host* al puerto del Pod.

```
kubectl port-forward first-pod 8080:80  
# Esto mapea el puerto 80 del Pod al puerto 8080 del  
localhost del host.
```

- **Mejor Práctica (Producción/Pruebas):** Para el acceso de usuarios, la opción ideal es a través del uso de **Services** en Kubernetes, que exponen el Pod de forma permanente y escalable, tema que se verá en la siguiente sesión.

Exposición de Aplicaciones en Kubernetes:

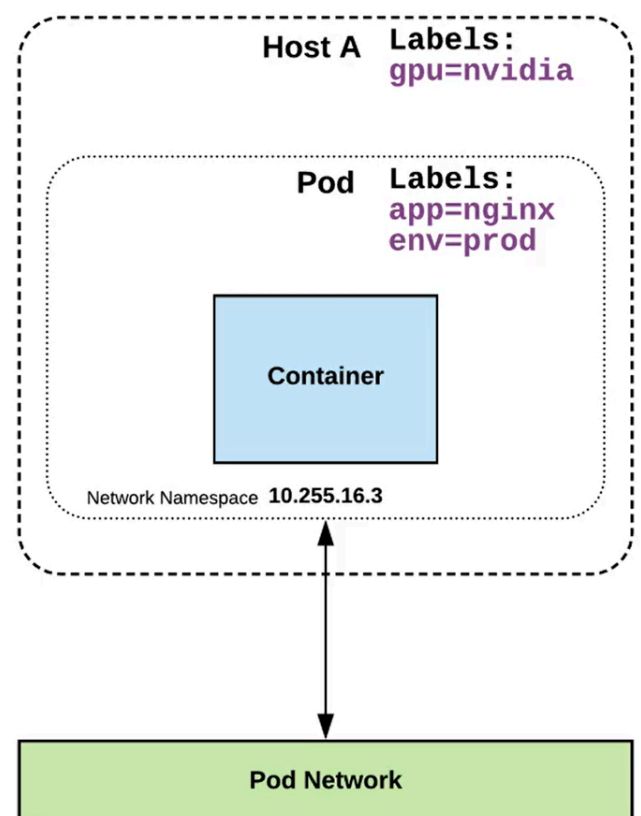
Etiquetas y Servicios

La forma fundamental de hacer que las aplicaciones desplegadas en **Pods** sean accesibles, ya sea internamente por otros servicios o externamente por usuarios, es mediante los **Servicios** (*Services*). Los Servicios proporcionan estabilidad y balanceo de carga, desacoplando la conectividad de la naturaleza volátil de los Pods. El mecanismo que conecta un Servicio a un Pod es la **Etiqueta** (*Label*).

I. Etiquetas (**Labels**): El Mecanismo de Selección

Las **Etiquetas** son metadatos que se asignan a los objetos de Kubernetes (Pods, Nodos, Services, etc.) para organizarlos y agruparlos según diferentes criterios.

- **Definición:** Son pares clave-valor que se definen dentro de la sección **metadata** de un objeto.
- **Importancia Clave:** Las etiquetas son el concepto más importante en Kubernetes para la gestión de recursos. Recursos superiores (como Services o Deployments) utilizan un **Selector de Etiquetas** (*Label Selector*) para identificar a qué Pods deben aplicar una política (ej. redireccionar el tráfico o escalarlos).
- **Ejemplo Visual de Etiquetado:** El esquema ilustra un Pod dentro del Host A. El Pod tiene etiquetas asignadas (**app=nginx**, **env=prod**), y el Host también puede tener etiquetas (ej. **gpu=nvidia**).



II. Servicios (**Services**): Exposición y Estabilidad

Un **Servicio** es una abstracción que define un conjunto lógico de Pods y una política estable para acceder a ellos. Los servicios resuelven la volatilidad de las direcciones IP de los Pods.

1. Características Fundamentales

- **Desacoplamiento:** Permite que los Pods escalen dinámicamente o se muevan entre Nodos sin afectar la conectividad, ya que el *endpoint* del servicio es estable.
- **Endpoint Estable:** Proporciona una dirección IP y un puerto que **nunca cambian**.
- **Balanceo de Carga:** Distribuye el tráfico de red de entrada entre todos los Pods que pertenecen al servicio.
- **Descubrimiento de Servicios:** Los servicios tienen un nombre DNS gestionado por Kubernetes que facilita la comunicación entre servicios internos.

2. Mecanismo del Selector de Etiquetas

El componente que conecta un Servicio a un Pod es el **Selector de Etiquetas** definido en la especificación del servicio.

- **Proceso:** El tráfico que llega al servicio es distribuido por **kube-proxy** a todos los Pods en el mismo *Namespace* que coincidan con las etiquetas definidas en el selector.

III. Tipos de Servicios y Diagramas de Exposición

A continuación, se detallan los tres tipos principales de servicios utilizados para exponer aplicaciones, con sus respectivos diagramas de arquitectura.

A. ClusterIP (Uso Interno)

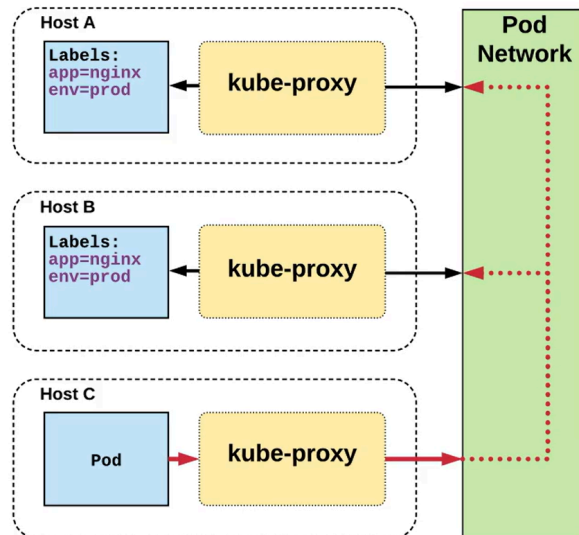
Este es el tipo de servicio que se crea por defecto.

- **Exposición:** Expone el servicio en una **dirección IP interna** (Cluster IP) dentro del clúster.
- **Acceso:** Solo accesible **internamente** (por otros Pods o servicios).

```

apiVersion: v1
kind: Service
metadata:
  name: example-prod
spec:
  selector:
    app: nginx
    env: prod
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80

```



- **Explicación del Esquema ClusterIP:** La imagen muestra la definición de un servicio ClusterIP (`kind: Service`, `type` implícito) con un selector (`app: nginx`, `env: prod`). El tráfico que entra al *ClusterIP* es enrutado internamente por el **kube-proxy** a los Pods que coinciden con las etiquetas (Host A y Host B). Este servicio reside completamente dentro de la **Pod Network** del clúster.

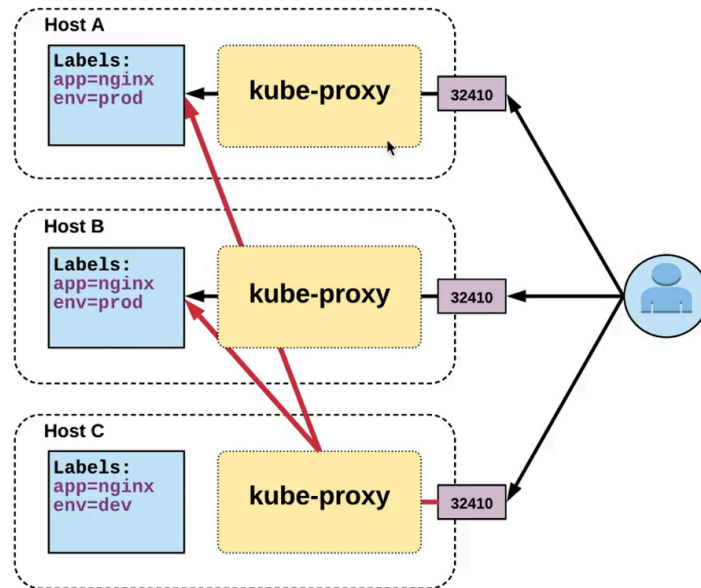
B. NodePort

El servicio NodePort expone el servicio en un puerto estático en todos los Nodos, permitiendo el acceso desde fuera del clúster.

- **Exposición:** Asigna y expone el servicio en un **puerto estático** y fijo (el *NodePort*, típicamente en el rango 30000-32767) en **todos los Nodos Trabajadores** del clúster.
- **Acceso:** Accesible **externamente** utilizando la IP de **cualquier Nodo** del clúster más el *NodePort* asignado (ej. http://<IP_Nodo>:31936).

nodes

```
apiVersion: v1
kind: Service
metadata:
  name: example-prod
spec:
  type: NodePort
  selector:
    app: nginx
    env: prod
  ports:
    - nodePort: 32410
      protocol: TCP
      port: 80
      targetPort: 80
```

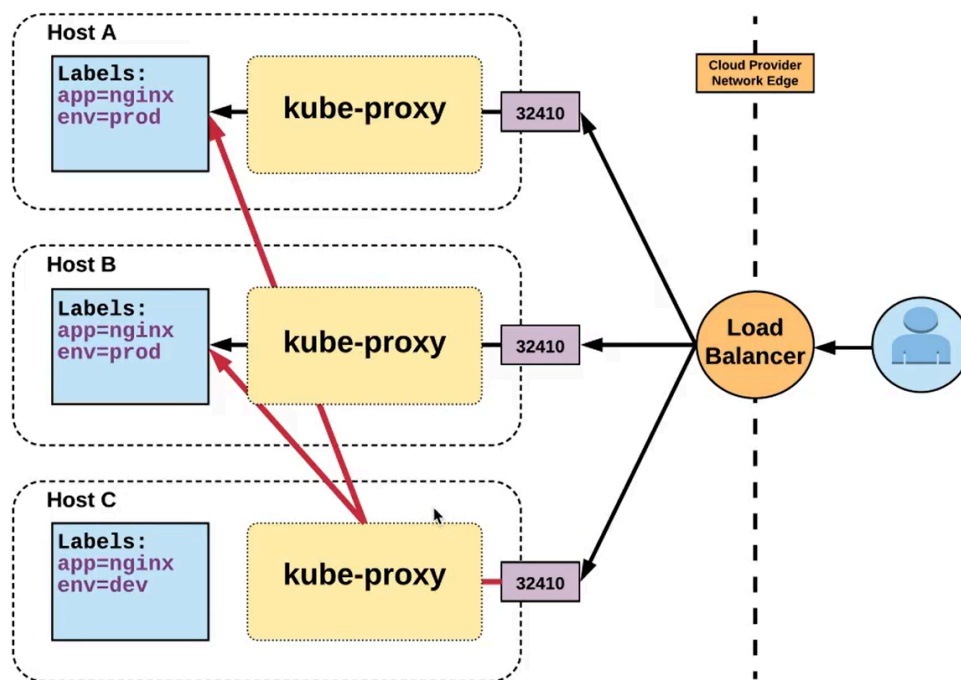


- **Explicación del Esquema NodePort:** La definición del servicio incluye explícitamente `type: NodePort`. El usuario externo accede al *NodePort* (32410 en el ejemplo) en la IP de cualquiera de los Nodos (A, B o C). El **kube-proxy** de ese Nodo recibe el tráfico y lo redirige a un Pod que coincida con el selector, incluso si ese Pod reside en otro Nodo.

C. LoadBalancer

El servicio LoadBalancer está diseñado para entornos en la nube, exponiendo el servicio a través de un balanceador de carga externo.

- **Exposición:** Es una extensión del NodePort, pero automáticamente aprovisiona un **Balanceador de Carga externo** (ofrecido por el proveedor de la nube).
- **Acceso:** El usuario accede a la IP pública del Balanceador de Carga. Este balanceador distribuye el tráfico a los NodePorts de los Nodos Trabajadores.



- **Explicación del Esquema LoadBalancer:** El tráfico externo es recibido por el **Load Balancer**. Este Balanceador de Carga actúa como punto de entrada público y luego distribuye las peticiones a los *NodePorts* (puertos estáticos) de los Nodos Trabajadores (A, B y C). De ahí, el **kube-proxy** toma el control y enruta el tráfico al Pod final.

IV. Procedimientos Prácticos de Despliegue de Servicios

Se utiliza el Pod base (`demoapp-v1`, con etiquetas `app: myapp` y `ver: 1`) para demostrar la exposición.

Paso 1: Despliegue de Pod y Verificación de Etiquetas

1. **Aplicar el Pod:** Se despliega el Pod con las etiquetas necesarias.

```
kubectl apply -f demoappv1.yaml
```

2. **Verificar Etiquetas y Estado:** Se comprueba que el Pod esté *running* y tenga las etiquetas que coinciden con los selectores del servicio.

```
kubectl get po --show-labels
```

Paso 2: Exposición con ClusterIP (Uso Interno)

1. **Aplicar la Definición del Servicio ClusterIP:** El selector (`app: myapp, ver: 1`) coincide con las etiquetas del Pod.

```
kubectl apply -f clusterip.yaml
```

2. **Verificar IP Interna:**

```
kubectl get svc # Muestra la Cluster IP asignada.
```

- a. *Para Acceder:* Solo desde dentro del clúster (ej. `minikube ssh` y `curl [Cluster IP]:8080`).

Paso 3: Exposición con NodePort (Uso Externo)

1. **Aplicar la Definición del Servicio NodePort:** Se establece `type: NodePort`.

```
kubectl apply -f nodeport.yaml
```

2. **Verificar NodePort Asignado:** El puerto de acceso externo se asigna automáticamente (rango alto).

```
kubectl get svc
```

3. **Acceder desde el Host (Externamente):** Se utiliza la IP del Nodo (Minikube) y el *NodePort*.

```
minikube ip # Obtiene la IP del Nodo.  
curl [IP_Nodo]:[NodePort_Asignado]
```

V. Buenas Prácticas y Riesgo de Volatilidad

- **Importancia del Servicio:** El uso de servicios es la forma **correcta** de comunicación, ya sea entre servicios internos o para exposición externa.
- **Estabilidad:** Los servicios son la clave de la estabilidad en Kubernetes, ya que las **IPs de los Pods son muy volátiles** y pueden cambiar si el Pod se reinicia o se mueve.
- **Etiquetas y Selectores:** Es fundamental que el selector de etiquetas del servicio coincida exactamente con las etiquetas de los Pods. Si el selector falla, el servicio no redireccionará tráfico.
- **NodePort y Acceso al Nodo:** El NodePort permite el acceso externo, pero requiere que el usuario tenga comunicación directa con los Nodos del clúster.

Exposición Avanzada de Aplicaciones: Kubernetes

Ingress

El **Ingress** es el método preferido para gestionar y enrutar el tráfico HTTP y HTTPS externo a los Servicios de Kubernetes. Proporciona una capa de abstracción de *routing* en la Capa 7 (Aplicación), permitiendo a los usuarios acceder a múltiples aplicaciones dentro del clúster utilizando una única dirección IP pública o nombre de *host*.

I. Fundamentos y Características de Ingress

Un **Ingress** es un objeto de la API de Kubernetes que gestiona el acceso externo a los servicios basándose en reglas definidas por el nombre de *host* y la ruta (*path*).

1. Características Clave

- **Punto de Entrada HTTP/S:** Actúa como el punto de entrada para el tráfico HTTP y HTTPS. Si una aplicación utiliza otro protocolo (ej. TCP crudo o UDP), Ingress no es una opción; en ese caso, se deben usar servicios **LoadBalancer** o **NodePort**.
- **Enrutamiento Basado en Reglas (Capa 7):** Define reglas que dirigen las solicitudes entrantes a diferentes servicios (*backend*) basándose en la **ruta URL** y el **nombre de host**.
- **Terminación de TLS:** Puede manejar la terminación de TLS (descifrar las solicitudes HTTPS entrantes) antes de dirigir el tráfico a los servicios de *backend* (que pueden ejecutarse internamente sobre HTTP).
- **Balanceo de Carga (Capa 7):** Proporciona balanceo de carga en la capa de aplicación, permitiendo dirigir el tráfico basándose en atributos HTTP/S, a diferencia de los servicios de Kubernetes que balancean en la Capa 4 (transporte).
- **Integración con Servicios:** Ingress **requiere** que el tráfico se dirija a un **Servicio** de Kubernetes (idealmente **ClusterIP**) y nunca directamente a un Pod.

2. Controlador de Ingress

Para que el objeto Ingress funcione, el clúster necesita tener instalado y en ejecución un **Controlador de Ingress**. Este controlador es el *proxy* inverso real (ej. Nginx, HAProxy o Traefik) que escucha en el puerto de acceso externo y aplica las reglas definidas.

- **Minikube:** En Minikube, el controlador de Ingress (por defecto Nginx) se habilita a través de un *addon*:

```
minikube addons enable ingress
```

II. Procedimiento Práctico: Despliegue de Ingress con Enrutamiento por Path

Para demostrar el enrutamiento avanzado, se despliegan dos versiones de una aplicación (**v1** y **v2**), cada una expuesta por su propio Servicio **ClusterIP**.

Paso 1: Despliegue de Pods y Servicios Internos

Se despliegan dos Pods y dos Servicios (ClusterIP) para ser utilizados como *backends* del Ingress.

1. **Pods con Etiquetado por Versión:** Se crean dos Pods, cada uno con una etiqueta de versión única (**ver: "1"** o **ver: "2"**) y su imagen de aplicación correspondiente.
 - **demoappv1.yaml** (Pod): **app: myapp**, **ver: "1"**
 - **demoappv2.yaml** (Pod): **app: myapp**, **ver: "2"**

Contenido del demoappv1.yaml:

```
apiVersion: v1
kind: Pod
metadata:
  name: demoapp-v1
  labels:
    app: myapp
    ver: "1"
spec:
  containers:
```

- name: demoapp
image: tuxotron/demoapp:v1
ports:
 - containerPort: 8080

Contenido del demoappv2.yaml:

```
apiVersion: v1
kind: Pod
metadata:
  name: demoapp-v2
  labels:
    app: myapp
    ver: "2"
spec:
  containers:
    - name: demoapp
      image: tuxotron/demoapp:v2
      ports:
        - containerPort: 8080
```

2. **Servicios ClusterIP Aislados:** Se crean dos Servicios ClusterIP, cada uno apuntando a una versión específica mediante su selector de etiquetas.

- o clusteripv1.yaml (Service): Selector app: myapp, ver: "1" \$\\implies\$ backend service-v1
- o clusteripv2.yaml (Service): Selector app: myapp, ver: "2" \$\\implies\$ backend service-v2

Contenido del clusteripv1.yaml:

```
apiVersion: v1
kind: Service
metadata:
  name: service-v1
spec:
  selector:
    app: myapp
```

```
    ver: "1"
  ports:
    - port: 8080
```

Contenido del clusteripv2.yaml:

```
apiVersion: v1
kind: Service
metadata:
  name: service-v2
spec:
  selector:
    app: myapp
    ver: "2"
  ports:
    - port: 8080
```

3. Aplicar Recursos (Declarativo):

```
kubectl apply -f clusteripv1.yaml -f clusteripv2.yaml -f
demoappv1.yaml -f demoappv2.yaml
```

4. **Verificación de Endpoints:** Se usa el comando `describe` para asegurar que cada servicio haya encontrado su respectivo Pod (Endpoint).

```
kubectl describe svc service-v1
```

Paso 2: Definición y Despliegue de Ingress (YAML)

El objeto Ingress define las reglas de enrutamiento necesarias para dirigir el tráfico externo a los servicios internos. Inicialmente, se define el objeto Ingress enfocándose en la primera regla de enrutamiento para la versión 1, para luego expandir la configuración.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
```

```
name: ingress-demoapp
annotations:
  nginx.ingress.kubernetes.io/rewrite-target: /$1
spec:
  rules:
    - host: demoapp.lab.local
      http:
        paths:
          - path: /v1
            pathType: Prefix
            backend:
              service:
                name: service-v1
                port:
                  number: 8080
```

- **apiVersion: networking.k8s.io/v1 & kind: Ingress:** Define el objeto de red.
- **annotations:** Utilizado para pasar opciones de configuración específicas al controlador de Ingress (en este caso Nginx). La anotación **rewrite-target: /\$1** es crítica para el *routing* basado en ruta, indicando al *proxy* Nginx que reescriba la URL internamente.
- **spec/rules/host: demoapp.lab.local:** La regla solo se aplica a las peticiones que lleguen con este nombre de *host*.
- **paths:** Se definen dos reglas de *routing* basadas en la ruta URL:
 - **path: /v1** (pathType: Prefix) se dirige al *backend* **service-v1** en el puerto **8080**. **Propósito de la Regla Inicial (/v1):**
 - Se comienza definiendo únicamente el *path* **/v1** para establecer el enrutamiento base.
 - Cualquier petición que llegue al **host: demoapp.lab.local** y contenga la ruta **/v1** se dirige al *backend* **service-v1**.

5. Despliegue del Ingress:

```
kubectl apply -f ingress.yaml
```

Demostración de la Necesidad del *Path*:

Petición sin *path* (`curl -H "Host: demoapp.lab.local"`)

`$(minikube ip)`): El Ingress responde con un error de "no encontrado" porque, aunque identifica el *host* correcto, **no tiene un *path* definido** para la petición raíz (/).

Petición con *path* (`curl -H "Host: demoapp.lab.local"`)

`$(minikube ip)/v1`): El Ingress **sí encuentra la regla** y redirige el tráfico al `service-v1`.

Expansión de Reglas para Soporte Multi-Versión (Añadiendo */v2*): Una vez probada la versión 1, se añade una segunda regla de *path* dentro de la misma especificación de Ingress:

```
- path: /v2
    pathType: Prefix
    backend:
      service:
        name: service-v2
        port:
          number: 8080
```

Razón Estratégica: Este patrón de enrutamiento por ruta es crucial para escenarios profesionales como:

- **Despliegues *Blue/Green* o *Canary*:** Permite mantener múltiples versiones de la aplicación activas simultáneamente (ej. */v1* estable y */v2* en prueba) para dirigir el tráfico de forma controlada o segmentada.

- **A/B Testing:** Dirigir usuarios a diferentes versiones de características.
- **APIs Versionadas:** Exponer diferentes versiones de una API REST.

Estructura Final del Ingress (Incluyendo /v2):

```
apiVersion: networking.k8s.io/v1

kind: Ingress
metadata:
  name: ingress-demoapp
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /$1
spec:
  rules:
    - host: demoapp.lab.local
      http:
        paths:
          - path: /v1
            pathType: Prefix
            backend:
              service:
                name: service-v1
                port:
                  number: 8080
          - path: /v2
            pathType: Prefix
            backend:
              service:
                name: service-v2
                port:
                  number: 8080
```

Aplicación y Prueba de la Versión 2: Se aplica el cambio (`kubectl apply -f ingress.yaml`) y se verifica la nueva regla.

```
curl -H "Host: demoapp.lab.local" $(minikube ip)/v2
```

Resultado: Respuesta de la Versión 2 de la aplicación.

V. Conclusiones

- **Ingress es el Estándar:** Ingress es la forma **ideal** de exponer aplicaciones **HTTP/HTTPS** que requieren enrutamiento avanzado, simplificando la gestión de múltiples *backends* (ej. versiones) bajo un único *endpoint* público.
- **Requisito del Servicio:** Ingress siempre debe dirigir el tráfico a un **Servicio** (idealmente **ClusterIP**) y nunca directamente a un Pod. El uso de ClusterIP es recomendable sobre NodePort, ya que evita abrir puertos adicionales y innecesarios en los Nodos.
- **Controladores Potentes:** Aunque el controlador por defecto de Kubernetes es Nginx (de los más sencillos), existen otros controladores más potentes y flexibles con más opciones de *routing* y configuración.